



Web Application External Penetration Testing Report

Security Assessment of Altoro Mutual (demo.testfire.net)

Client: Lab / Demo Environment

Application: Altoro Mutual

Target URL: <http://demo.testfire.net>

Tester: Sachin S(CU23BCA0047A)

Assessment Period: 01 May 2026 – 02 May 2026

Classification: Confidential

Table of Contents

1. Executive Summary	1
1.1 Assessment Overview	1
1.2 Key Findings	2
1.3 Overall Risk Rating	2
2. Scope and Methodology	3
2.1 Scope of Assessment	3
2.2 In-Scope Assets	3
2.3 Out-of-Scope Items	4
2.4 Rules of Engagement	4
2.5 Testing Methodology (OWASP)	5
2.6 Tools and Techniques	5
3. Findings Summary by OWASP Category	6
3.1 A01: Broken Access Control	6
3.2 A02: Cryptographic Failures	6
3.3 A03: Injection	7
3.4 A05: Security Misconfiguration	7
3.5 Summary of All Findings	8
4. Detailed Findings	9
4.1 W-001: Weak TLS Configuration	9
4.2 W-002: LOGJAM Vulnerability	10
4.3 W-003: LUCKY13 Vulnerability	11
4.4 W-004: Missing Security Headers	12
4.5 W-005: CSRF Vulnerability	13
4.6 W-006: Information Disclosure	14
4.7 W-007: Authentication Bypass (SQL Injection)	15
4.8 W-008: Unauthorized Fund Transfer	16
5. Risk Ratings	17
5.1 Risk Classification Criteria	17
5.2 Severity Distribution	18
6. Remediation Recommendations	19
6.1 Immediate Actions	19
6.2 Short-Term Improvements	20
6.3 Long-Term Security Enhancements	21
7. Appendix	22
7.1 Tools Used	22
7.2 Evidence and Logs	23
7.3 References	24

1. Executive Summary

A web application External penetration test was conducted on **Altoro Mutual (demo.testfire.net)** between **01/05/2026 – 02/05/2026**.

The assessment focused on externally exposed services, web application behaviour, SSL/TLS configuration, and common OWASP Top 10 risks.

Overall Security Posture: MEDIUM

Several security weaknesses were identified, primarily in:

- Cryptographic configuration (TLS weaknesses)
- Missing security controls (headers, CSRF protection)
- Legacy protocol support

These issues could allow attackers to downgrade encryption, perform cryptographic attacks, or exploit application-level weaknesses.

A critical authentication bypass vulnerability was identified, allowing attackers to gain administrative access without valid credentials. This was further exploited to perform unauthorized financial transactions.

2. Scope and Methodology

Target:

- demo.testfire.net
- IP: 65.61.137.117

Testing Type:

- External Web Application Pentest

Methodology:

- Based on OWASP Testing Guide v4 & OWASP Top 10 (2021)

Tools Used:

- Nmap
- Gobuster
- WhatWeb
- testssl.sh
- Nmap NSE scripts

Testing Areas:

- Network Reconnaissance
- Service Enumeration
- SSL/TLS Analysis
- Web Application Security

3. Findings Summary by OWASP Category

OWASP Category	Finding Count	Highest Severity
A01 - Broken Access Control	2	Critical
A02 - Cryptographic Failures	3	High
A03 - Injection	1	Critical
A07 - Security Misconfiguration	3	Medium

4. Detailed Findings

W-001: Weak TLS Configuration (Deprecated Protocols Enabled)

- **Severity:** High
- **OWASP:** A02: Cryptographic Failures
- **Affected:** <https://demo.testfire.net>

Description:

The server supports deprecated TLS versions:

- TLS 1.0
- TLS 1.1

Impact:

Attackers can:

- Perform downgrade attacks
- Exploit known weaknesses (BEAST, etc.)

Evidence:

TLS 1 offered (deprecated)

TLS 1.1 offered (deprecated)

Remediation:

- Disable TLS 1.0 and TLS 1.1
- Enforce TLS 1.2+ or TLS 1.3

W-002: LOGJAM Vulnerability (Weak Diffie-Hellman)

- **Severity:** High
- **CVE:** CVE-2015-4000
- **OWASP:** A02

Description:

Server uses weak 1024-bit DH group.

Evidence:

LOGJAM: VULNERABLE (1024-bit DH group)

Impact:

- Possible man-in-the-middle attack
- Weak key exchange security

Remediation:

- Use $\geq$ 2048-bit DH parameters
- Prefer ECDHE ciphers

W-003: LUCKY13 Vulnerability (CBC Cipher Use)

- **Severity:** Medium
- **CVE:** CVE-2013-0169

Description:

Server supports CBC-mode ciphers, vulnerable to timing attacks.

Evidence:

potentially VULNERABLE (CBC ciphers used)

Impact:

- Possible plaintext recovery

Remediation:

- Disable CBC ciphers
- Use AEAD (GCM) ciphers only

W-004: Missing Security Headers

- **Severity:** Medium
- **OWASP:** A05: Security Misconfiguration

Description:

Server lacks important HTTP security headers.

Evidence:

Strict Transport Security: not offered

Security headers: --

Impact:

- Increased risk of:
 - XSS
 - Clickjacking
 - MITM attacks

Remediation:

Add headers:

- Content-Security-Policy
- X-Frame-Options
- X-Content-Type-Options
- HSTS

W-005: Potential CSRF Vulnerabilities

- **Severity:** Medium
- **OWASP:** A01: Broken Access Control

Description:

Multiple forms lack CSRF protection.

Evidence:

Form action: /search.jsp

Form action: doSubscribe

Form action: sendFeedback

Impact:

- Attackers can perform actions on behalf of users

Remediation:

- Implement CSRF tokens
 - Validate origin/referrer headers
-

W-006: Information Disclosure via Service Banner

- **Severity:** Low

Description:

Server reveals technology stack:

Apache-Coyote/1.1

Java (JSESSIONID)

Impact:

- Helps attackers fingerprint system

Remediation:

- Remove or obfuscate server headers
-

W-007: Authentication Bypass via SQL Injection

- **Severity:** Critical
 - **CVSS 3.1:** 9.8 (AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H)
 - **OWASP:** A03:2021 – Injection
 - **CWE:** CWE-89 – SQL Injection
-

Affected Endpoint

- /login.jsp

Description

The login functionality is vulnerable to SQL Injection.

An attacker can bypass authentication by injecting SQL payloads into the username and password fields.

The application does not properly validate or sanitize user input before passing it to the backend database.

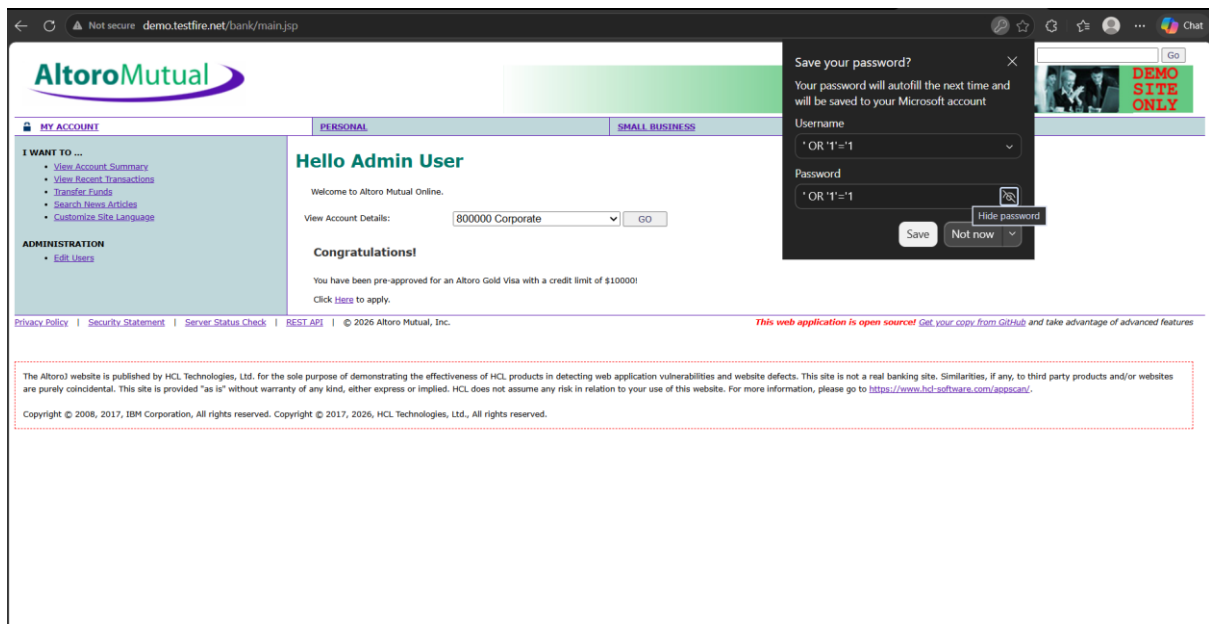
Proof of Concept (PoC)**Payload used:**

' OR '1'='1

Result:

- Authentication bypass achieved
- Logged in as **Admin User** without valid credentials

Evidence



Impact

An attacker can:

- Bypass authentication completely
- Gain unauthorized admin access
- View sensitive data
- Perform privileged actions

This leads to full account compromise.

Remediation

- Use parameterized queries / prepared statements
- Implement input validation
- Use ORM frameworks (Hibernate, etc.)
- Add WAF rules for SQL injection detection

W-008: Unauthorized Fund Transfer (Business Logic Abuse)

- **Severity:** Critical
- **CVSS:** 9.1
- **OWASP:** A01:2021 – Broken Access Control

Affected Endpoint

- /bank/doTransfer

Description

After bypassing authentication, the attacker was able to perform a **fund transfer operation without proper authorization checks**.

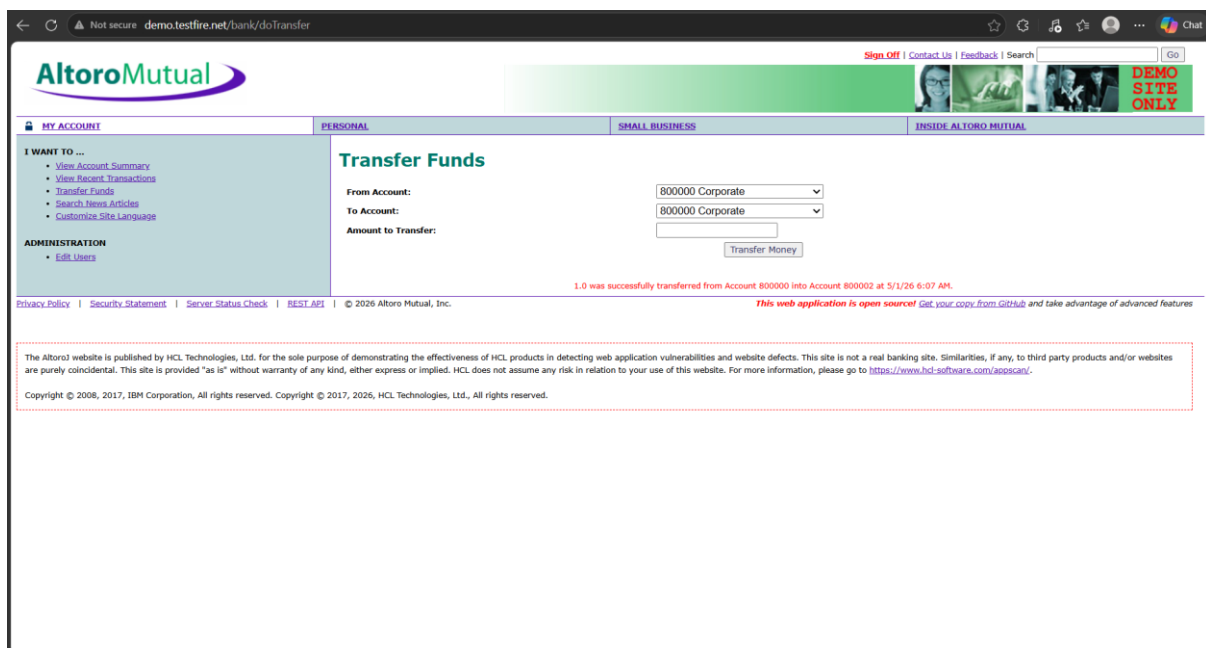
The application does not validate:

- User identity properly
- Account ownership
- Transaction authorization

Proof of Concept

- Access gained via SQL Injection
- Navigated to **Transfer Funds**
- Executed transaction:
 - From: 800000
 - To: 800002
 - Amount: 1

Evidence



Impact

An attacker can:

- Transfer funds from any account
- Perform unauthorized financial transactions
- Cause financial loss
- Compromise integrity of banking system

Remediation

- Enforce **server-side authorization checks**
- Validate account ownership before transactions
- Implement **multi-factor authentication (MFA)**
- Add **transaction signing / OTP verification**

5. Risk Ratings

Severity	Description
Critical	None
High	TLS Weaknesses (LOGJAM, Deprecated TLS)
Medium	CSRF, Missing Headers, CBC Ciphers
Low	Banner Disclosure

6. Remediation Recommendations

- 1.Disable TLS 1.0 / 1.1
- 2 Upgrade cryptographic configuration (ECDHE + AES-GCM)
- 3 Implement full HTTP security headers
- 4 Add CSRF protection across all forms
- 5 Harden server configuration (hide banners)
- 6 Regularly patch and update components

7. FULL WORKFLOW

```
(kali@kali)-[~]
└─$ mkdir demo.testfire

(kali@kali)-[~]
└─$ ls
demo.testfire  Desktop  Documents  Downloads  Music  Pictures  Public  Templates  Videos

(kali@kali)-[~]
└─$ cd demo.testfire

(kali@kali)-[~/demo.testfire]
└─$ mkdir pentestings

(kali@kali)-[~/demo.testfire]
└─$ cd pentestings

(kali@kali)-[~/demo.testfire/pentestings]
└─$ nano scope.txt
```

First, I made directory demo.testfire , inside that I made another folder named pentestings where I have added all of my findings.

```
(kali@kali)-[~/demo.testfire/pentestings]
$ cat scope.txt
Client: Lab / Demo
Engagement Type: External Network Penetration Test
SI Windows

In-Scope:
- Hostname: demo.testfire.net
- Resolved IP(s): (to be discovered)
- All TCP/UDP services exposed on this host

Out-of-Scope:
- Any other domains/IPs not owned by this demo

Rules of Engagement:
- No DoS or stress testing
- No password spraying against unrelated systems
- Non-destructive proof-of-concept only

(kali@kali)-[~/demo.testfire/pentestings]
$ nano notes.md

(kali@kali)-[~/demo.testfire/pentestings]
$ cat notes.md
# External Network Pentest - demo.testfire.net

## Targets
- Hostname: demo.testfire.net
- Resolved IP(s): (to be filled)

## Timeline
- Recon start:
- Scanning start:
- Enumeration:
- Vulnerability analysis:
- Reporting:

## High-Level Observations
- (Bullet points as you go)

## Evidence Index
- (List of screenshots and text files)
```

I have Created scope.txt and notes.md In order to keep track of my External Pentesting

```

(kali㉿kali)-[~/demo.testfire/pentestings]
$ ls
notes.md  scope.txt

(kali㉿kali)-[~/demo.testfire/pentestings]
$ cat notes.md
# External Network Pentest - demo.testfire.net

## Targets
- Hostname: demo.testfire.net
- Resolved IP(s): (to be filled)

## Timeline
- Recon start:01/05/2026
- Scanning start:01/05/2026
- Enumeration:02/05/2026
- Vulnerability analysis:01/05/2026
- Reporting:02/05/2026

## High-Level Observations
- (Bullet points as you go)

## Evidence Index
- (List of screenshots and text files)

```

Here You can see I have given every details about the pentesting which Im doing on demo.testfire.net

```

(kali㉿kali)-[~/demo.testfire/pentestings]
$ dig +short demo.testfire.net | tee ip_demo.txt
65.61.137.117

(kali㉿kali)-[~/demo.testfire/pentestings]
$ ping demo.testfire.net
PING demo.testfire.net (65.61.137.117) 56(84) bytes of data.
64 bytes from 65.61.137.117: icmp_seq=1 ttl=64 time=306 ms
64 bytes from 65.61.137.117: icmp_seq=2 ttl=64 time=330 ms
64 bytes from 65.61.137.117: icmp_seq=3 ttl=64 time=353 ms
^C
— demo.testfire.net ping statistics —
4 packets transmitted, 3 received, 25% packet loss, time 3002ms
rtt min/avg/max/mdev = 306.483/329.902/353.139/19.047 ms

(kali㉿kali)-[~/demo.testfire/pentestings]
$ ls
ip_demo.txt  notes.md  scope.txt

```

I have ran - **dig +short demo.testfire.net**

This command resolves the domain name to its IP address. So, demo.testfire.net → 65.61.137.117

DNS is working correctly.

And at last given ip_demo.txt so that result get save into that file automatically

```
(kali㉿kali)-[~/demo.testfire/pentestings]
$ for t in A AAAA NS MX TXT; do
  echo "=== $t records ===" | tee -a dns_demo_testfire.txt
  dig demo.testfire.net $t +short | tee -a dns_demo_testfire.txt
done
=== A records ===
65.61.137.117
=== AAAA records ===
=== NS records ===
=== MX records ===
=== TXT records ===
```

This commands means

Loop through DNS record types:

- **A** → IPv4 address
- **AAAA** → IPv6 address
- **NS** → Name servers
- **MX** → Mail servers
- **TXT** → Text records (SPF, verification, etc.)

For each type, it queries DNS using dig

+short → shows only clean results (no extra info)

== A records ==
65.61.137.117

A record found

→ Domain resolves to IPv4: **65.61.137.117**

```

(kali㉿kali)-[~/demo.testfire/pentestings]
$ whois 65.61.137.117 | tee whois_demo_ip.txt

#
# ARIN WHOIS data and services are subject to the Terms of Use
# available at: https://www.arin.net/resources/registry/whois/tou/
# File System
# If you see inaccuracies in the results, please report at
# https://www.arin.net/resources/registry/whois/inaccuracy_reporting/
#
# Copyright 1997-2026, American Registry for Internet Numbers, Ltd.
#

# start

NetRange:      65.61.128.0 - 65.61.191.255
CIDR:          65.61.128.0/18
NetName:       RSPC-NET-4
NetHandle:     NET-65-61-128-0-1
Parent:        NET65 (NET-65-0-0-0-0)
NetType:       Direct Allocation
OriginAS:
Organization:  Rackspace Hosting (RACKS-8)
RegDate:       2002-11-01
Updated:       2012-02-24
Ref:           https://rdap.arin.net/registry/ip/65.61.128.0

OrgName:       Rackspace Hosting
OrgId:         RACKS-8
Address:       1718 Dry Creek Way
Address:       Ste 115
City:          San Antonio
StateProv:     TX
PostalCode:    78259-1837
Country:       US
RegDate:       2010-03-29
Updated:       2025-01-31
Ref:           https://rdap.arin.net/registry/entity/RACKS-8

OrgTechHandle: HANSE157-ARIN
OrgTechName:   Hansell, Chris
OrgTechPhone:  +1-210-312-4000
OrgTechEmail:  hostmaster@rackspace.com
OrgTechRef:    https://rdap.arin.net/registry/entity/HANSE157-ARIN

OrgTechHandle: IPADM17-ARIN
OrgTechName:   IPADMIN
OrgTechPhone:  +1-210-312-4000
OrgTechEmail:  hostmaster@rackspace.com
OrgTechRef:    https://rdap.arin.net/registry/entity/IPADM17-ARIN

OrgTechHandle: ZR9-ARIN
OrgTechName:   Rackspace, com
OrgTechPhone:  +1-210-312-4000
OrgTechEmail:  hostmaster@rackspace.com
OrgTechRef:    https://rdap.arin.net/registry/entity/ZR9-ARIN

```

Here I ran `whois 65.61.137.117 | tee whois_demo_ip.txt`

The command `whois 65.61.137.117 | tee whois_demo_ip.txt` checks who owns the IP address and saves the result to a file. The output shows that the IP belongs to **Rackspace Hosting**, meaning the

website is hosted on a cloud provider, not its own server. It also gives details like IP range, location (USA), and contact info. In short, you confirmed the hosting provider and infrastructure of the target.

```
(kali@kali)-[~/demo.testfire/pentestings]
$ nmap -sS -sV -Pn --top-ports 1000 65.61.137.117 -oA nmap-top_1000
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-01 09:51 -0400
Nmap scan report for 65.61.137.117
Host is up (0.31s latency).
Not shown: 996 filtered tcp ports (no-response)
PORT      STATE SERVICE VERSION
80/tcp    open  http      Apache Tomcat/Coyote JSP engine 1.1
443/tcp   open  ssl/http   Apache Tomcat/Coyote JSP engine 1.1
8080/tcp  open  http      Apache Tomcat/Coyote JSP engine 1.1
8443/tcp  closed https-alt
```

Here I ran `nmap -sS -sV -Pn --top-ports 1000 65.61.137.117 -oA nmap_top_1000`

-sS → Stealth SYN scan (fast and less detectable)

-sV → Detect service versions

-Pn → Skip host discovery (assume host is alive)

--top-ports 1000 → Scan most common 1000 ports

-oA → Save results in files

The server is running **Apache Tomcat (Java-based web server)**

Web services are available on:

- Port 80 (normal web)
- Port 443 (secure web)
- Port 8080 (often admin/test panel)

```
(kali㉿kali)-[~/demo.testfire/pentestings]
$ nmap -sS -sV -Pn -p- 65.61.137.117 -oA nmap_full_tcp
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-01 09:56 -0400
Stats: 0:13:01 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 2.67% done; ETC: 18:04 (7:55:00 remaining)
Stats: 0:19:35 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 2.94% done; ETC: 21:02 (10:46:25 remaining)
Stats: 0:55:00 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 1.47% done; ETC: 00:17 (61:26:37 remaining)
Stats: 0:58:48 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 1.52% done; ETC: 02:34 (63:39:52 remaining)
Stats: 1:01:19 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 1.55% done; ETC: 04:00 (65:03:33 remaining)
Stats: 1:02:58 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 1.57% done; ETC: 04:55 (65:56:22 remaining)
Stats: 1:02:59 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 1.57% done; ETC: 04:56 (65:57:49 remaining)
Stats: 1:03:01 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 1.57% done; ETC: 04:57 (65:58:12 remaining)
Stats: 1:03:34 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 1.57% done; ETC: 05:15 (66:15:43 remaining)
Stats: 1:03:36 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 1.57% done; ETC: 05:16 (66:16:29 remaining)
Stats: 1:05:50 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 1.60% done; ETC: 06:27 (67:25:43 remaining)
Stats: 1:06:20 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 1.61% done; ETC: 06:43 (67:40:43 remaining)
Stats: 1:08:14 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 1.63% done; ETC: 07:41 (68:36:46 remaining)
Stats: 1:11:59 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 1.68% done; ETC: 09:31 (70:23:06 remaining)
Stats: 1:12:56 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 1.69% done; ETC: 09:58 (70:49:19 remaining)
Stats: 1:13:49 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 1.70% done; ETC: 10:23 (71:13:16 remaining)
Stats: 1:14:29 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 1.71% done; ETC: 10:41 (71:30:56 remaining)
Stats: 1:18:57 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
```

I ran The command `nmap -sS -sV -Pn 65.61.137.117 -oA nmap_full_tcp`

performs a more detailed port scan of the target. Here, `-sS` does a stealth SYN scan, `-sV` tries to detect service versions, and `-Pn` skips the host discovery step assuming the target is online. Unlike your previous scan, this one checks **all 65,535 TCP ports**, not just the top 1000, which is why it's taking a long time. The output you see (timing stats and percentages) simply shows scan progress and estimated completion time. In short, this command is doing a **full, in-depth scan of every TCP port** on the target and saving the results for later analysis.

```

(kali@kali)-[~/demo.testfire/pentestings]
$ whatweb http://demo.testfire.net --log-brief whatweb_http80.txt
http://demo.testfire.net [200 OK] Apache, Cookies[JSESSIONID], Country[UNITED STATES][US], HTTPServer[Apache-Coyote/1.1], HttpOnly[JSESSIONID], IP[65.61.137.117], Java, Title[Altoro Mutual]

(kali@kali)-[~/demo.testfire/pentestings]
$
File System
(kali@kali)-[~/demo.testfire/pentestings]
$ ls
dns_demo_testfire.txt  nmap_full_tcp.gnmap  nmap_full_tcp.xml  nmap-top_1000.nmap  notes.md  whatweb_http80.txt
ip_demo.txt           nmap_full_tcp.nmap  nmap-top_1000.gnmap  nmap-top_1000.xml  scope.txt  whois_demo_ip.txt

(kali@kali)-[~/demo.testfire/pentestings]
$ whatweb https://demo.testfire.net --log-brief whatweb_http443.txt
https://demo.testfire.net [200 OK] Apache, Cookies[JSESSIONID], Country[UNITED STATES][US], HTTPServer[Apache-Coyote/1.1], HttpOnly[JSESSIONID], IP[65.61.137.117], Java, Title[Altoro Mutual]

```

This command shows `whatweb http://demo.testfire.net --log-brief whatweb_http80.txt` scans the website to identify technologies used and saves a short result to a file. The output shows the site is live (200 OK), running on **Apache-Coyote (Tomcat)**, using **Java**, and sets a JSESSIONID cookie (typical for Java apps). It also reveals the page title “**Altoro Mutual**” and the server location (US). You repeated the same scan for HTTPS (https://...) and saved it to another file. In short, this command helps you fingerprint the web application stack and confirm it’s a Java-based Tomcat application, which is useful for targeting specific web vulnerabilities.

```

(kali@kali)-[~/demo.testfire/pentestings]
$ testssl
Command 'testssl' not found, but can be installed with:
sudo apt install testssl.sh
Do you want to install it? (N/y)y
sudo apt install testssl.sh
[sudo] password for kali:
The following packages were automatically installed and are no longer required:
  libicu76      libipc-run-perl  libteamctl0  openjdk-21-jre  postgresql-common-dev
  libio-pty-perl  libsmdutf31     libunibreak6  openjdk-21-jre-headless  python3-pysnmp4
Use 'sudo apt autoremove' to remove them.

Installing:
  testssl.sh

Installing dependencies:
  libengine-gost-openssl

Summary:
  Upgrading: 0, Installing: 2, Removing: 0, Not Upgrading: 53
  Download size: 1,372 kB
  Space needed: 5,446 kB / 57.6 GB available

Continue? [Y/n] y
Get:1 http://http.kali.org/kali kali-rolling/main amd64 libengine-gost-openssl amd64 3.0.3+git20251212.7f6d61a-2 [525 kB]
Get:2 http://http.kali.org/kali kali-rolling/main amd64 testssl.sh all 3.2.2+dfsg-1 [847 kB]
Fetched 1,372 kB in 1s (1,676 kB/s)
Selecting previously unselected package libengine-gost-openssl.
(Reading database... 433764 files and directories currently installed.)
Preparing to unpack ../libengine-gost-openssl_3.0.3+git20251212.7f6d61a-2_amd64.deb...
Unpacking libengine-gost-openssl (3.0.3+git20251212.7f6d61a-2)...
Selecting previously unselected package testssl.sh.
Preparing to unpack ../testssl.sh_3.2.2+dfsg-1_all.deb...
Unpacking testssl.sh (3.2.2+dfsg-1)...
Setting up testssl.sh (3.2.2+dfsg-1)...
Setting up libengine-gost-openssl (3.0.3+git20251212.7f6d61a-2)...
Processing triggers for man-db (2.13.1-1)...
Processing triggers for kali-menu (2026.2.2)...
Scanning processes...
Scanning candidates...
Scanning linux images...

```

I have successfully installed **testssl.sh**, which is used to analyze SSL/TLS security (like HTTPS weaknesses, ciphers, vulnerabilities). You're now ready to run SSL security scans on the target.

```
(kali㉿kali)-[~/demo.testfire/pentestings]
-$ testssl https://demo.testfire.net | tee testssl_demo.txt

#####
testssl version 3.2.2 from https://testssl.sh/

This program is free software. Distribution and modification under
GPLv2 permitted. USAGE w/o ANY WARRANTY. USE IT AT YOUR OWN RISK!

Please file bugs @ https://testssl.sh/bugs/
#####

Using OpenSSL 3.6.1 (Mar 18 2026) [~103 ciphers]
on kali:/usr/bin/openssl

Start 2026-05-01 10:20:32      —> 65.61.137.117:443 (demo.testfire.net) <—

rDNS (65.61.137.117):  --
Service detected:      HTTP

Testing protocols via sockets except NPN+ALPN

SSLv2      not offered (OK)
SSLv3      not offered (OK)
TLS 1      offered (deprecated)
TLS 1.1    offered (deprecated)
TLS 1.2    offered (OK)
TLS 1.3    not offered and downgraded to a weaker protocol
NPN/SPDY   not offered
ALPN/HTTP2 not offered

Testing cipher categories

NULL ciphers (no encryption)          not offered (OK)
Anonymous NULL Ciphers (no authentication) not offered (OK)
Export ciphers (w/o ADH+NULL)         not offered (OK)
LOW: 64 Bit + DES, RC[2,4], MD5 (w/o export) not offered (OK)
Triple DES Ciphers / IDEA             not offered
Obsoleted CBC ciphers (AES, ARIA etc.) offered
Strong encryption (AEAD ciphers) with no FS not offered
Forward Secrecy strong encryption (AEAD ciphers) offered (OK)
```

The command **testssl https://demo.testfire.net | tee testssl_demo.txt** runs an SSL/TLS security scan on the target website and saves the results to a file. It checks which encryption protocols and cipher suites the server supports. From the output, the server does **not support SSLv2/SSLv3 (good)** but still allows **TLS 1.0 and TLS 1.1 (deprecated and insecure)**, while **TLS 1.2 is supported (good)** and **TLS 1.3 is not available**. It also shows that weak ciphers are mostly disabled, but some **obsolete ciphers are still offered**, which is a security concern. In short, the scan reveals that the server's HTTPS configuration is **partially secure but outdated**, with older protocols and some weak encryption still enabled.

```

--(kali@kali)-[~/demo.testfire/pentestings]
$ testssl --max-ssl-fail 10 https://demo.testfire.net
usr/bin/testssl: unrecognized option "--max-ssl-fail"

"testssl [options] <URI>" or "testssl <options>"

testssl <option>", where <option> is mostly standalone and one of:

--help                what you're looking at
-b, --banner          displays banner + version of testssl
-v, --version         same as previous
-V, --local [pattern] pretty print all local ciphers (of openssl only). If search pattern supplied: it
is an                an ignore case word pattern of cipher hexcode or any other string in its name, k
or bits

testssl [options] <URI>", where <URI> is:

<URI>                host|host:port|URL|URL:port port 443 is default, URL can only contain HTTPS as
a protocol

and [options] is/are:

-t, --starttls <protocol> Does a run against a STARTTLS enabled service which is one of ftp, smtp, lmtpt, p
p3, imap,
--xmpphost <to_domain>    xmpp, xmpp-server, telnet, ldap, nntp, sieve, postgres, mysql
For STARTTLS xmpp or xmpp-server checks it supplies the domainname (like SNI)
--mx <domain/host>        Tests MX records from high to low priority (STARTTLS, port 25)
--file/-il <fname>       Mass testing option: Reads one testssl.sh command line per line from <fname>.
Can be combined with --serial or --parallel. Implicitly turns on "--warnings bat
h".
--mode <serial|parallel> Text format 1: Comments via # allowed, EOF signals end of <fname>
the latter)              Text format 2: nmap output in greppable format (-oG), 1 port per line allowed
--warnings <batch|off>    Mass testing to be done serial (default) or parallel (--parallel is shortcut for
kips warnings            "batch" doesn't continue when a testing error is encountered, off continues and
--connect-timeout <seconds> useful to avoid hangers. Max <seconds> to wait for the TCP socket connect to ret
rn
--openssl-timeout <seconds> useful to avoid hangers. Max <seconds> to wait before openssl connect will be te
minated

single check as <options> ("testssl URI" does everything except -E and -g):
-e, --each-cipher        checks each local cipher remotely
-E, --cipher-per-proto   checks those per protocol
-s, --std, --categories  tests standard cipher categories by strength
-f, --fs, --forward-secrecy checks forward secrecy settings
-p, --protocols          checks TLS/SSL protocols (including ALPN/HTTP2 and SPDY)
-g, --grease             tests several server implementation bugs like GREASE and size limitations
-S, --server-defaults    displays the server's default picks and certificate info
-P, --server-preference  displays the server's picks: protocol+cipher

```

I was not getting required results so I ran `testssl --fast https://demo.testfire.net`

Testing vulnerabilities

```
Heartbleed (CVE-2014-0160)      not vulnerable (OK), no heartbeat extension
CCS (CVE-2014-0224)            not vulnerable (OK)
Ticketbleed (CVE-2016-9244), experiment. not vulnerable (OK), no session ticket extension
ROBOT                           Server does not support any cipher suites that use RSA key transport supported (OK)
Secure Renegotiation (RFC 5746) not vulnerable (OK) -- mitigated (1 successful renege within 10 in 33s(timed out))
Secure Client-Initiated Renegotiation
CRIME, TLS (CVE-2012-4929)      not vulnerable (OK)
BREACH (CVE-2013-3587)         no gzip/deflate/comp/br HTTP compression (OK) - only supplied "/" tested
POODLE, SSL (CVE-2014-3566)     not vulnerable (OK), no SSLv3 support
TLS_FALLBACK_SCSV (RFC 7507)   Downgrade attack prevention NOT supported
SWEET32 (CVE-2016-2183, CVE-2016-6329) not vulnerable (OK)
FREAK (CVE-2015-0204)          not vulnerable (OK)
DROWN (CVE-2016-0800, CVE-2016-0703) not vulnerable on this host and port (OK)
                                make sure you don't use this certificate elsewhere with SSLv2 enabled services, see
                                https://search.censys.io/search?resource=hosts&virtual_hosts=INCLUDE&q=B0EAC225501DB594A9639F9E718BC036759B9F492C7C729A60C8473568393382
LOGJAM (CVE-2015-4000), experimental
                                VULNERABLE (NOT ok): common prime: RFC2409/Oakley Group 2 (1024 bits), but no DH EXPORT ciphers
                                TLS1: ECDHE-RSA-AES256-SHA DHE-RSA-AES256-SHA ECDHE-RSA-AES128-SHA DHE-RSA-AES128-SHA
                                VULNERABLE -- but also supports higher protocols TLSv1.1 TLSv1.2 (likely mitigated)
LUCKY13 (CVE-2013-0169), experimental
                                potentially VULNERABLE, uses obsolete cipher block chaining ciphers with TLS, see server prefs.
Winshock (CVE-2014-6321), experimental
RC4 (CVE-2013-2566, CVE-2015-2808) not vulnerable (OK) - CAMELLIA or ECDHE_RSA GCM ciphers found
                                Local problem: No RC4 Ciphers configured in /usr/bin/openssl
```

Running client simulations (HTTP) via sockets

Browser	Protocol	Cipher Suite Name (OpenSSL)	Forward Secrecy
Android 7.0 (native)	TLSv1.2	ECDSA-RSA-AES128-GCM-SHA256	256 bit ECDH (P-256)
Android 8.1 (native)	TLSv1.2	ECDSA-RSA-AES128-GCM-SHA256	256 bit ECDH (P-256)
Android 9.0 (native)	TLSv1.2	ECDSA-RSA-AES128-GCM-SHA256	256 bit ECDH (P-256)
Android 10.0 (native)	TLSv1.2	ECDSA-RSA-AES128-GCM-SHA256	256 bit ECDH (P-256)
Android 11/12 (native)	TLSv1.2	ECDSA-RSA-AES128-GCM-SHA256	256 bit ECDH (P-256)
Android 13/14 (native)	TLSv1.2	ECDSA-RSA-AES128-GCM-SHA256	256 bit ECDH (P-256)
Android 15 (native)	TLSv1.2	ECDSA-RSA-AES128-GCM-SHA256	256 bit ECDH (P-256)

```
(kali@kali) [~/demo.testfire/pentestings]
$ cat testssl_demo.txt | grep VULN
LOGJAM (CVE-2015-4000), experimental
                                VULNERABLE (NOT ok): common prime: RFC2409/Oakley Group 2 (1024 bits),
                                VULNERABLE -- but also supports higher protocols TLSv1.1 TLSv1.2 (likely mitigated)
LUCKY13 (CVE-2013-0169), experimental
                                potentially VULNERABLE, uses obsolete cipher block chaining ciphers with TLS, see server prefs.
```

The command `cat testssl_demo.txt | grep VULN` filters your SSL scan results to show only lines containing the word “VULN” (vulnerabilities). Instead of reading the full report, it quickly highlights security issues. The output shows the server is **vulnerable to LOGJAM (weak 1024-bit Diffie-Hellman group)** and **LUCKY13 (related to CBC cipher weaknesses)**. Even though stronger protocols like TLS 1.2 are supported, the presence of weak configurations means the server is still considered vulnerable. In short, this command helps you quickly extract and identify critical SSL vulnerabilities from the scan results.

```

(kali@kali)-[~/demo.testfire/pentestings]
$ nmap -sV --script vuln -Pn 65.61.137.117 -oA nmap_vuln_full
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-01 10:41 -0400
Nmap scan report for 65.61.137.117
Host is up (0.25s latency).
Not shown: 997 filtered tcp ports (no-response)
PORT      STATE SERVICE      VERSION
80/tcp    open  tcpwrapped
|_http-aspnet-debug: ERROR: Script execution failed (use -d to debug)
|_http-server-header: Apache-Coyote/1.1
|_http-vuln-cve2014-3704: ERROR: Script execution failed (use -d to debug)
|_http-dombased-xss: Couldn't find any DOM based XSS.
|_http-stored-xss: Couldn't find any stored XSS vulnerabilities.
|_http-csrf:
|_Spidering limited to: maxdepth=3; maxpagecount=20; withinhost=65.61.137.117
|_Found the following possible CSRF vulnerabilities:

|_Path: http://65.61.137.117:80/
|_Form id: frmsearch
|_Form action: /search.jsp

|_Path: http://65.61.137.117:80/index.jsp?content=business_deposit.htm
|_Form id: frmsearch
|_Form action: /search.jsp

|_Path: http://65.61.137.117:80/index.jsp?content=business_lending.htm
|_Form id: frmsearch
|_Form action: /search.jsp

|_Path: http://65.61.137.117:80/status_check.jsp
|_Form id: frmsearch
|_Form action: /search.jsp

|_Path: http://65.61.137.117:80/status_check.jsp
|_Form id: frmjsonsubmit
|_Form action: javascript:checkSiteStatus('AltoroMutual')

|_Path: http://65.61.137.117:80/index.jsp?content=business_cards.htm
|_Form id: frmsearch
|_Form action: /search.jsp

|_Path: http://65.61.137.117:80/index.jsp?content=business_other.htm
|_Form id: frmsearch
|_Form action: /search.jsp

|_Path: http://65.61.137.117:80/index.jsp?content=personal_checking.htm
|_Form id: frmsearch
|_Form action: /search.jsp

```

The command `nmap -sV --script vuln -Pn 65.61.137.117 -oA nmap_vuln_full` runs an advanced Nmap scan that not only detects service versions (-sV) but also executes built-in vulnerability detection scripts (--script vuln). The -Pn option skips host discovery, and -oA saves the output in multiple formats. From the results, port **80 (HTTP)** is open and running **Apache-Coyote (Tomcat)**. Some scripts failed to execute (which is common), but the scan identified **possible CSRF vulnerabilities** in multiple web application paths (like /search.jsp forms). It did not find DOM-based or stored XSS issues. In short, this command helps automatically detect common web vulnerabilities, and here it highlights **potential CSRF weaknesses** in the application.

```

VULNERABLE:
Diffie-Hellman Key Exchange Insufficient Group Strength
State: VULNERABLE
  Transport Layer Security (TLS) services that use Diffie-Hellman groups
  of insufficient strength, especially those using one of a few commonly
  shared groups, may be susceptible to passive eavesdropping attacks.
Check results:
Weak DH GROUP 1
  Cipher Suite: TLS_DHE_RSA_WITH_AES_128_CBC_SHA
  Modulus Type: Safe prime
  Modulus Source: RFC2409/Oakley Group 2
  Modulus Length: 1024
  Generator Length: 8
  Public Key Length: 1024
References:
  https://weakdh.org
http-server-header: Apache-Coyote/1.1
http-sql-injection: ERROR: Script execution failed (use -d to debug)
80/tcp open  tcpwrapped
http-enum:
  /login.jsp: Possible admin folder
  /login.jsp: Login page
http-vuln-cve2014-3704: ERROR: Script execution failed (use -d to debug)
http-aspnet-debug: ERROR: Script execution failed (use -d to debug)
http-server-header: Apache-Coyote/1.1

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 131.13 seconds

```

```

(kali@kali) ~/demo.testfire/pentestings
$ nuclei -l targets_http.txt -o nuclei_demo.txt

File System
v3.8.0
projectdiscovery.io

[WRN] Found 1 templates with runtime error (use -validate flag for further examination)
[INF] Current nuclei version: v3.8.0 (latest)
[INF] Current nuclei-templates version: v10.4.2 (latest)
[INF] New templates added in latest release: 121
[INF] Templates loaded for current scan: 10095
[INF] Executing 10079 signed templates from projectdiscovery/nuclei-templates
[WRN] Loading 16 unsigned templates for scan. Use with caution.
[INF] Targets loaded for current scan: 1
[INF] Templates clustered: 2281 (Reduced 2154 Requests)
[INF] Using Interactsh Server: oast.me
[swagger-api] [http] [info] https://demo.testfire.net/swagger/index.html [paths="/swagger/index.html"]
[waf-detect:apache-generic] [http] [info] https://demo.testfire.net
[deprecated-tls:tls_1.1] [ssl] [info] demo.testfire.net:443 ["tls11"]
[weak-cipher-suites:tls-1.0] [ssl] [low] demo.testfire.net:443 ["tls10 TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA"]
[tls-version] [ssl] [info] demo.testfire.net:443 ["tls10"]
[deprecated-tls:tls_1.0] [ssl] [info] demo.testfire.net:443 ["tls10"]
[weak-cipher-suites:tls-1.1] [ssl] [low] demo.testfire.net:443 ["tls11 TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA"]
[tls-version] [ssl] [info] demo.testfire.net:443 ["tls11"]
[tls-version] [ssl] [info] demo.testfire.net:443 ["tls12"]
[missing-cookie-samesite-strict] [http] [info] https://demo.testfire.net ["JSESSIONID=E60D0BCBE77B2DCE1F25B900BECA0
ACF; Path=/; Secure; HttpOnly"]
[http-missing-security-headers:missing-content-type] [http] [info] https://demo.testfire.net
[http-missing-security-headers:permissions-policy] [http] [info] https://demo.testfire.net
[http-missing-security-headers:x-content-type-options] [http] [info] https://demo.testfire.net
[http-missing-security-headers:x-permitted-cross-domain-policies] [http] [info] https://demo.testfire.net
[http-missing-security-headers:referrer-policy] [http] [info] https://demo.testfire.net
[http-missing-security-headers:cross-origin-embedder-policy] [http] [info] https://demo.testfire.net
[http-missing-security-headers:strict-transport-security] [http] [info] https://demo.testfire.net
[http-missing-security-headers:content-security-policy] [http] [info] https://demo.testfire.net
[http-missing-security-headers:x-frame-options] [http] [info] https://demo.testfire.net
[http-missing-security-headers:clear-site-data] [http] [info] https://demo.testfire.net
[http-missing-security-headers:cross-origin-opener-policy] [http] [info] https://demo.testfire.net
[http-missing-security-headers:cross-origin-resource-policy] [http] [info] https://demo.testfire.net
[apache-detect] [http] [info] https://demo.testfire.net ["Apache-Coyote/1.1"]

```

The command `nuclei -t targets_http.txt -o nuclei_demo.txt` runs the **Nuclei vulnerability scanner** against targets listed in `targets_http.txt` and saves the findings to `nuclei_demo.txt`. Nuclei uses a large set of community templates to automatically check for known vulnerabilities and misconfigurations. From the output, it loaded thousands of templates and scanned your target (`demo.testfire.net`). The results show multiple issues such as **deprecated TLS versions (TLS 1.0/1.1)**, **weak cipher suites**, **missing security headers** (like CSP, HSTS, X-Frame-Options), and **cookie misconfiguration (missing SameSite)**. It also detected technologies like **Apache Tomcat** and found an exposed **Swagger API endpoint**. In short, this command performs automated vulnerability scanning and highlights several **security misconfigurations and outdated settings** on the target.

```
[weak-cipher-suites:tls-1.1] [ssl] [low] demo.testfire.net:443 ["[tls11 TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA]"]
[tls-version] [ssl] [info] demo.testfire.net:443 ["tls11"]
[tls-version] [ssl] [info] demo.testfire.net:443 ["tls12"]
[missing-cookie-samesite-strict] [http] [info] https://demo.testfire.net ["JSESSIONID=E60D0BCBE77B2DCE1F25B9008ECA0ACF; Path=/; Secure; HttpOnly"]
[http-missing-security-headers:missing-content-type] [http] [info] https://demo.testfire.net
[http-missing-security-headers:permissions-policy] [http] [info] https://demo.testfire.net
[http-missing-security-headers:x-content-type-options] [http] [info] https://demo.testfire.net
[http-missing-security-headers:x-permitted-cross-domain-policies] [http] [info] https://demo.testfire.net
[http-missing-security-headers:referrer-policy] [http] [info] https://demo.testfire.net
[http-missing-security-headers:cross-origin-embedder-policy] [http] [info] https://demo.testfire.net
[http-missing-security-headers:strict-transport-security] [http] [info] https://demo.testfire.net
[http-missing-security-headers:content-security-policy] [http] [info] https://demo.testfire.net
[http-missing-security-headers:x-frame-options] [http] [info] https://demo.testfire.net
[http-missing-security-headers:clear-site-data] [http] [info] https://demo.testfire.net
[http-missing-security-headers:cross-origin-opener-policy] [http] [info] https://demo.testfire.net
[http-missing-security-headers:cross-origin-resource-policy] [http] [info] https://demo.testfire.net
[apache-detect] [http] [info] https://demo.testfire.net ["Apache-Coyote/1.1"]
[INF] Skipped demo.testfire.net:4040 from target list as found unresponsive permanently: cause="port closed or filtered" address=demo.testfire.net:4040 chain="i/o timeout; got err while executing https://demo.testfire.net:4040/jobs/"
[caa-fingerprint] [dns] [info] demo.testfire.net
[ssl-issuer] [ssl] [info] demo.testfire.net:443 ["Sectigo Limited"]
[ssl-dns-names] [ssl] [info] demo.testfire.net:443 ["demo.testfire.net"]
[INF] Scan completed in 9m. 26 matches found.
```

```
(kali@kali)-[~/demo.testfire/pentestings]
$ ls
dns_demo_testfire.txt  nmap_full_tcp.xml  nmap_vuln_full.gnmap  nuclei_demo.txt  whatweb_http443.txt
ip_demo.txt           nmap-top_1000.gnmap  nmap_vuln_full.nmap   scope.txt        whatweb_http80.txt
nmap_full_tcp.gnmap    nmap-top_1000.nmap  nmap_vuln_full.xml    targets_http.txt  whois_demo_ip.txt
nmap_full_tcp.nmap     nmap-top_1000.xml   notes.md              testssl_demo.txt
```

Here are all of my findings Regarding this External Pentesting Report

8. Appendix

[..\OneDrive\Documents\windows-linux\pentestings](https://OneDrive/Documents/windows-linux/pentestings)

(sir if anyway this link exposes my other file directories please avoid it 😊)

— End of Report —